SpacerScope User Manual (English Version)

1. Introduction

What is SpacerScope?

SpacerScope is a high-performance off-target analysis tool for **CRISPR/Cas genome editing** applications. It is designed to address the high computational cost, slow response time, and difficulty of balancing speed and accuracy that traditional methods often face when working with large reference genomes, especially complex plant genomes.

The core idea of SpacerScope is to divide off-target searching into three stages: **site extraction, fast pre-filtering, and exact verification**. The program first extracts candidate spacer sites that satisfy PAM constraints from the query sequence and the reference genome. It then applies different efficient encoding strategies for different search tasks. In **mismatch searching**, it uses compact **2-bit sequence encoding** together with batch scanning to quickly evaluate Hamming distance. In **insertion/deletion searching**, sequences are first encoded into multi-channel binary features, and **channel filtering plus batch mask fusion** is used to greatly reduce the candidate set before exact alignment is performed only on the remaining small subset. For indel verification, SpacerScope uses **right-end-anchored edit-distance dynamic programming** instead of simple local alignment, making it better suited to CRISPR spacer scenarios where insertions, deletions, and terminal constraints coexist.

As a result, SpacerScope does not rely solely on traditional full-sequence alignment. Instead, it combines **binary-encoding-driven high-efficiency pre-filtering** with **exact edit-distance verification**, significantly reducing overall search cost while preserving result completeness and accuracy. Practical tests show that SpacerScope can deliver fast, stable, and high-accuracy gRNA screening and off-target analysis even in complex genome backgrounds.

Core Features of SpacerScope

1. **Comprehensive off-target detection capability**
   Under user-defined constraints such as PAM, mismatch count, and indel count, SpacerScope can systematically search candidate sites that satisfy the conditions and preserve result completeness and decision accuracy as much as possible during exact verification.

2. **Support for both mismatch and indel off-target analysis**
   SpacerScope supports not only conventional **base substitution (mismatch)** detection, but also off-target identification involving **insertions** and **deletions**, covering both common and more complex CRISPR/Cas off-target types.
3. **Efficient binary encoding and pre-filtering**
   The program uses specially designed binary representations and pre-screening strategies for different tasks:
   - In mismatch searching, compact **2-bit encoding** and batch scanning are used to accelerate large-scale Hamming-distance calculation.
   - In indel searching, multi-channel encoding, binary filtering, and batch mask fusion significantly reduce the number of candidates that need exact alignment.
     This "filter fast first, verify exactly later" design substantially improves overall runtime efficiency.
4. **Suitable for large-scale genome analysis**
   Through optimization of memory access, parallel scheduling, and candidate filtering workflows, SpacerScope maintains good computational efficiency on large reference genomes and is especially suitable for heavy-load scenarios such as complex plant genomes.
5. **Multiple input and analysis workflows**
   SpacerScope supports:
   - Direct input of FASTA query sequences
   - Extraction of target sequences from annotation files for analysis
   - Standard mode and low-memory cut mode
     In cut mode, the reference genome can be streamed chromosome by chromosome on machines with limited memory resources.
6. **Cross-platform compatibility**
   SpacerScope supports **Windows**, **Linux**, and **Mac**. It can be used either as a command-line tool or through `spacerscope-web` in a browser. Due to the specifics of macOS, users need to compile and package it themselves.
7. **Parallel execution and low-level optimization**
   The program supports multithreaded execution and can take advantage of instruction-set optimizations such as **AVX2 / SSE4.1** on supported hardware and build environments to further improve runtime performance.
8. **Adaptable to multiple CRISPR systems**
   Users can define custom **PAM sequences** and their relative orientation, allowing adaptation to different nuclease systems and experimental design requirements.

---

2. Program Components

The current release usually includes two executable files:

- `spacerscope`: main program
- `spacerscope-web`: Web GUI shell that internally invokes `spacerscope` through a subprocess

Their relationship is as follows:

- `spacerscope` performs all actual computation and can run independently.

- `spacerscope-web` provides a browser-based GUI and also makes network access more convenient. It is not an independent search engine and must be used together with `spacerscope`.

---

3. Environment Requirements

Supported Operating Systems

- Windows x64
- Linux x86_64
- Mac (requires manual compilation)

CPU Instruction Sets

Recommended CPU support:

- AVX2
- SSE4.1

On Linux, you can check with:

```
lscpu | grep -E 'avx2|sse4_1' | sed 's/^[[:space:]]*//'
```

The program can still run on machines without AVX2 support, but performance may be lower.

Runtime Dependencies

Core dependencies:

- OpenMP
- zlib

If you use `spacerscope-web`, no extra software is required other than a browser. You only need to make sure that the `spacerscope` executable can be found either in the same directory or through the specified path.

---

4. Running Modes

SpacerScope currently supports 4 main modes:

Mode	Input Method	Memory Strategy	Description
`fasta`	Query FASTA + Reference FASTA	Standard mode	Use FASTA query sequences directly
`fasta-cut`	Query FASTA + Reference FASTA	Low-memory mode	Stream the reference sequence chromosome by chromosome

Mode	Input Method	Memory Strategy	Description
`anno`	Annotation file + gene ID + Reference FASTA	Standard mode	Extract query sequences from annotations and then run
`anno-cut`	Annotation file + gene ID + Reference FASTA	Low-memory mode	Annotation extraction + chromosome-wise low-memory processing

Difference Between Standard Mode and Cut Mode

- **Standard mode**: the entire reference genome is loaded into memory, and performance is usually higher.
- **Cut mode**: the reference genome is processed chromosome by chromosome, resulting in lower peak memory usage and making it more suitable for memory-constrained machines.

Peak Memory Estimation

At startup, the program automatically estimates peak memory usage based on the reference FASTA and asks the user for confirmation. To skip this check, use:

```
-y
```

or

```
--yes
```

or

```
--skip-memory-check
```

- In standard mode, peak usage is mainly related to the **total genome size**
- In cut mode, peak usage is mainly related to the **size of the longest chromosome**

---

5. Command-Line Program: `spacerscope`

General Usage

```
spacerscope fasta <options>
spacerscope fasta-cut <options>
spacerscope anno <options>
spacerscope anno-cut <options>
spacerscope --report-tsv <file> --html-output <file>
```

Show help:

```
spacerscope --help
spacerscope help fasta
spacerscope help anno
```

Show version:

```
spacerscope --version
```

Common Parameters

Parameter	Description	Default
`-R, --ref <file>`	Reference genome FASTA file	Required
`-o, --output <file>`	Output TSV file	Required
`--pam <string>`	Primary PAM sequence	`NGG`
`--alt-pam <string>`	Alternative PAM sequence	`NAG`
`--spacer-len <int>`	Spacer length	`20`
`--direction <up|down>`	Direction of the spacer relative to the PAM	`up`
`-m, --mismatch <int>`	Maximum number of mismatches	`4`
`-i, --indel <int>`	Maximum number of indels	`2`
`--dust-threshold <int>`	DUST low-complexity filtering threshold	`12`
`-t, --threads <int>`	Number of threads	Number of logical cores
`-y, --yes`	Skip startup memory confirmation	Off
`--skip-memory-check`	Skip startup memory confirmation	Off
`--raw-output`	Output an unscored 7-column TSV	Off
`--progress-format <text|jsonl>`	Progress output format	`text`
`--html <0|1>`	Whether to generate `<output>.html`	`1`
`-ht <0|1>`	Alias of `--html`	`1`
`--html-output <file>`	Specify the HTML output file	Empty
`--report-tsv <file>`	Generate HTML only from an existing scored TSV	Empty

Parameters for `fasta / fasta-cut` Mode

Parameter	Description
`-I, --query <file>`	Query FASTA file

Examples

```
spacerscope fasta --query q.fa --ref ref.fa -o out.tsv
```

```
spacerscope fasta-cut --query q.fa --ref ref.fa -o out.tsv --yes
```

Parameters for `anno / anno-cut` Mode

Parameter	Description
`-A, --annotation <file>`	Annotation file (GFF/GFF3)
`--geneID <id>`	The **exact gene ID** in the annotation file
`--seq-type <gene|mrna|cds>`	Sequence level to extract, default is `gene`

Examples

```
spacerscope anno --annotation genes.gff3 --geneID gene:Gene1 --seq-type cds --ref ref.fa
-o out.tsv
```

```
spacerscope anno-cut --annotation genes.gff3 --geneID gene:Gene1 --seq-type gene --ref
ref.fa -o out.tsv --yes
```

Important Limitations of `anno` Mode

In the current version, `anno / anno-cut` has already been integrated into the C++ main program, but its annotation compatibility strategy is **released according to legacy conventions** and is not a universal parser for arbitrary GFF3 files.

Please note:

1. `--geneID` must match the **gene ID** in the annotation file exactly; it is not a free-form gene symbol.
2. `mrna/cds` extraction follows legacy conventions: transcript association depends on the `GeneX-mRNA` **naming relationship**.
3. For generic GFF3 files using styles such as `Parent=transcript:...`, the current version **does not guarantee** full compatibility in `mrna/cds` mode.

If your annotation file format is complex or inconsistent, it is recommended to:

- Extract the target gene sequence manually first
- Then run in `fasta` or `fasta-cut` mode

`--raw-output` and HTML Reports

In the current version:

- Standard output: **9-column scored TSV**
- `--raw-output`: **7-column unscored merged TSV**

The columns in files generated by `--raw-output` are:

Column Name
QueryName
QuerySeq
TargetName
TargetSeq
Distance
SearchType
GenomicFrequency

Note:

- `--raw-output` and `--html-output` **cannot be used together**
- HTML reports can only be generated from scored TSV files

Generate HTML Only from an Existing TSV

If you already have a scored TSV, you can generate HTML separately:

```
spacerscope --report-tsv out.tsv --html-output out.html
```

---

6. Web Version (GUI): `spacerscope-web`

Suitable Scenarios

`spacerscope-web` is the local web shell provided in the current version. It is suitable for:

- Linux servers without a desktop environment
- Users who prefer configuring parameters and viewing results through a browser
- Access scenarios using SSH port forwarding to the local service

How to Start

```
spacerscope-web
```

Default listening address:

```
127.0.0.1:8787
```

Common parameters:

Parameter	Description	Default
`--bind <host>`	Bind address	`127.0.0.1`
`--port <int>`	Listening port	`8787`
`--workspace-root <dir>`	Web working directory	`spacerscope-web` under the system temp directory
`--spacerscope-bin <path>`	Specify the path to the `spacerscope` executable	Auto-detected in the same directory
`-h, --help`	Show help	-
`-V, --version`	Show version	-

Examples

```
spacerscope-web --bind 127.0.0.1 --port 8787
spacerscope-web
```

Access

Open in a browser:

```
http://127.0.0.1:8787
```

If you use it on a remote Linux server, SSH port forwarding is recommended:

```
ssh -L 8787:127.0.0.1:8787 user@server
```

Then open the following in your local browser:

```
http://127.0.0.1:8787
```

Web Version Notes

The web interface supports:

- `fasta / fasta-cut / anno / anno-cut`
- Server-path input mode
- File upload mode
- Chinese/English language switching
- Real-time logs
- Real-time progress
- TSV / HTML / stdout / stderr downloads

6.5 Security Notice

The current `spacerscope-web` v1 **has no authentication mechanism**.

Therefore:

- By default, it should only be bound to `127.0.0.1`
- If it is bound to `0.0.0.0` or another LAN address, it should only be used on a **trusted network**
- Direct exposure to the public internet is not recommended

---

7. Output Files

Default Main Output (scored TSV)

The default output is a **9-column scored TSV**:

Column Name	Meaning
`QueryName`	Query site name
`QuerySeq`	Query site sequence
`TargetName`	Off-target site name
`TargetSeq`	Off-target site sequence
`Distance`	Edit distance
`SearchType`	Match type: `Exact` / `Mismatch` / `Indel`
`GenomicFrequency`	Number of occurrences of the sequence in the genome
`Score`	Off-target score
`Details`	Alignment details

HTML Report

If HTML output is enabled, the program additionally generates:

```
<output>.html
```

Or the file you specify with `--html-output`.

The HTML report includes:

- Summary statistics for each query
- Distribution of exact matches, mismatches, and indels
- Detailed result table

Raw TSV

When `--raw-output` is used, the program outputs a **7-column unscored TSV**, which does not include:

- `Score`
- `Details`

At the same time, no HTML report is generated.

---

8. Parameter and Runtime Recommendations

General Recommendations

- In most cases, the default parameters are already usable.
- Unless there is a clear need, it is not recommended to increase `--indel` arbitrarily.
- It is recommended to start with the default parameters and then fine-tune them as needed.

About `--indel`

Increasing `--indel` will noticeably increase computational cost.

Typical recommendations:

- `--indel 0`: exact matches + mismatches only
- `--indel 1~2`: recommended range for typical use
- Higher `--indel`: try only when it is truly necessary

About `--threads`

The program supports multithreading, but a larger thread count is not always better. Actual speed depends on memory bandwidth, NUMA behavior, machine architecture, and other factors.

Recommendations:

- Start by testing with the default thread count or a commonly effective high-performance thread count for the machine

- Benchmark on the target server to determine the best value

8.4 About Cut Mode

The purpose of cut mode is to **reduce peak memory usage**; it is not always faster. When memory is sufficient, standard mode is usually more direct. When memory is constrained, cut mode is safer.

---

9. Notes and Known Limitations

1. It is recommended that all related files be placed under **English-only paths** when using the program.
2. It is not recommended to run the entire genome directly as the query FASTA.
3. It is recommended to use a **complete continuous gene sequence** as the query file (for example, in `fasta` mode). If mRNA or CDS-level sequences are used directly as query sequences, splicing may cause the designed gRNA sequence not to exist in the genome, which may appear as zero exact-match sites or exact-match sites that are not located on the target gene.
4. For annotation files:
   - The current `anno` mode is not a universal parser for arbitrary GFF3 files
   - `mrna/cds` mode especially depends on legacy annotation conventions
5. Because annotation file formats are not uniform across datasets, it is recommended to extract the **gene sequence** and use **fasta mode**.
6. `spacerscope-web` v1 has no authentication and should not be exposed directly to the public internet.
7. `--progress-format jsonl` is mainly intended for the web shell and programmatic invocation. Ordinary users should usually use the default `text` format.
8. When selecting the final gRNA, confirm whether all exact-match sites are located on the **target gene**.
9. Ensure the software has sufficient permissions during use.

---

10. Quick Command Reference

FASTA Mode

```
spacerscope fasta --query query.fa --ref genome.fa -o result.tsv
```

FASTA-CUT Mode

```
spacerscope fasta-cut --query query.fa --ref genome.fa -o result.tsv --yes
```

ANNO Mode

```
spacerscope anno --annotation genes.gff3 --geneID gene:Gene1 --seq-type gene --ref genome.fa -o result.tsv
```

ANNO-CUT Mode

```
spacerscope anno-cut --annotation genes.gff3 --geneID gene:Gene1 --seq-type cds --ref genome.fa -o result.tsv --yes
```

Generate HTML Report Only

```
spacerscope --report-tsv result.tsv --html-output result.html
```

Start the Web Shell

```
spacerscope-web --bind 127.0.0.1 --port 8787
```